

A Biologically-Plausible Framework for Online Mining Spatio-Temporal Sequential Patterns from Event Data Streams

Piotr S. Maciag

piotr.maciag@pw.edu.pl

Institute of Computer Science, Warsaw University of Technology
Warsaw, Poland

Abstract

Mining spatio-temporal sequential patterns from event data is important for identifying recurring dependencies in domains such as crime analysis, mobility, and environmental monitoring. This paper proposes a biologically inspired framework for the online discovery of such patterns using Spiking Neural Networks (SNNs). The proposed method represents incoming events by their type, location, and occurrence time, maps them onto a spatial grid, and constructs an SNN whose structure reflects spatial proximity between event occurrences. We posit an analogy between unsupervised Spike-Time Dependent Plasticity (STDP) and unsupervised pattern mining in data mining: temporal regularities in event occurrences can be encoded as strengthened synaptic connections. Building on this principle, we leverage an STDP-inspired learning mechanism to develop online algorithms for mining significant spatio-temporal sequential patterns directly from the evolving SNN structure. The approach is evaluated on a real data stream of social activities using both quantitative and qualitative analyses. Quantitatively, we compare the proposed framework with baseline methods, while qualitatively, we employ visual analysis of the discovered patterns using developed software.

CCS Concepts

• **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

spatio-temporal data streams, sequential pattern mining, spiking neural networks, online pattern mining, spike-time dependent plasticity

ACM Reference Format:

Piotr S. Maciag. 2018. A Biologically-Plausible Framework for Online Mining Spatio-Temporal Sequential Patterns from Event Data Streams. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Mining frequent patterns from Spatio-Temporal (ST) event data has been long attracting attention of researchers. Such patterns can naturally represent recurring phenomena in e.g., crime or environmental analysis. For example, given a dataset of crime instances, one could be interested in revealing significant sequences of crime types, instances of which occur in spatio-temporal proximity [1, 10].

In this paper, we propose a biologically-inspired framework to continuously mine sequential patterns from spatio-temporal event data. Specifically, our approach is based on spiking neural networks and the Spike-Time Dependent Plasticity (STDP) rule to modify synaptical weights in such networks. In the proposed framework, a user first provides an ST dataset of events, each of which has been assigned a location (i.e., in a two-dimensional space), an occurrence time and an event type. Subsequently, ST space of the dataset is transformed into a 4 dimensional grid according to the user-given spatial and temporal cell resolutions. The grid is 4 dimensional as it encompasses two spatial dimensions (e.g., longitude and latitude), one temporal dimension and one dimension for event types representations. The grid is next used to construct a spiking neural network taught in an unsupervised way using STPD-like rules to reveal sequential patterns between event types.

Our approach differs significantly from previous works attempting to extract various types of patterns (such as sequential [6], cascading [14] or propagation [15]) from spatio-temporal data. While these and similar approaches are based on either (expensive) candidate pattern generation and evaluation strategies or employ graph-based mining algorithms, we deduct significant pattern from the structure of a taught spiking neural network. For example, the STS-Miner algorithm for mining spatio-temporal sequential patterns introduced by Huang et al. [6], starts the discovery process from singular event types which are recursively extended and evaluated to sequences of many event types. For the evaluation of such sequences Huang et al. proposed a new measure called sequence index. In contrast, to avoid costly and extensive iterative generation and evaluation of candidate pattern, we infer the significant sequential patterns based on strength of synaptical connections between neuron emitting spikes based on once processed input dataset.

1.1 Contributions

1.2 The Structure of the Article

The structure of the work is as follows. In Section 2, we review the related work. We split the review into two parts: (i) related to mining patterns from spatio-temporal event data, and (ii) biological foundations of spiking neural networks adapted in this work. In Section 3, we describe the proposed framework. Section 4 describes the developed learning algorithms of the neural network and the

pattern inference mechanisms. In Section 5, we offer the analysis of the proposed algorithms (in terms of time and space complexity) as well as give some of its further extensions. Section 6 gives the results of experiments using real datasets. Finally, in Section 7 we discuss the achieved results and concludes the work.

2 Related Work

2.1 Mining Spatio-Temporal Dependency Rules

In our work, we are interested in discovering STDRs from event data defined in a similar sense as by Huang et al. [6]. Specifically, they proposed a framework for mining such patterns that utilizes a depth-first recursive strategy of generating candidate patterns from already discovered significant (aka. strong) patterns. A sequential pattern was defined there as a sequence of event types $\vec{s} = F_1 \rightarrow \dots \rightarrow F_l$, which expresses an occurrence dependency among these types, such that, within their spatio-temporal neighbourhoods, instances of type F_1 either attract or repel occurrences of type F_2 . In turn, instances of type F_2 attract or repel occurrences of the subsequent event type, and so on, up to type F_k . The strength of the relationship between consecutive event type is quantified using the Participation Ratio (PR) measure, while the overall significance of a sequence $\vec{s}$ is evaluated using the Sequence Index (SI), defined as the minimum of all corresponding PR values. The developed pattern mining framework was implemented by the recursive STS-Miner and Slice-STS-Miner algorithms, where the latter allowed to mine pattern for large dataset that can not fit entirely into main memory. The approach presented by Huang et al. [6] was dedicated for *off-line mining* (i.e., in a non-continuous environment), where the entire dataset is available from the beginning.

Maciag et al. [12] offered an alternative approach to mine STSPs in an off-line mode. First, they proposed to discover *closed* STSPs which constitute a concise and informative representation of all STSPs. They also offered a more efficient algorithm that employs a breadth-first strategy to discover such patterns.

Mohan et al.[14] and Moosavi et al.[15] proposed alternative spatio-temporal pattern representations beyond STSPs. CSTPs model partially ordered cascades of event types, enabling richer dependency structures, while propagation patterns represent frequent embedded subtrees capturing spatio-temporal relationships between event instances. Other research has investigated mining STSPs from more complex objects, such as regions [2, 3] and trajectories [8] or mining sequential patterns from uncertain ST data [9].

2.2 Biologically-Plausible Pattern Discovery

3 Proposed Solution

3.1 Data Representation

Let $D = \{e_1, e_2, \dots\}$ denote the continuous stream of discrete event instances and $F = \{F_1, F_2, \dots, F_K\}$ the corresponding set of event types. Each event instance $e \in D$ is characterized by its spatial location ($e.location$), time of occurrence ($e.time$), and has an event type $e.F \in F$. The example of such datastream is presented by us in Fig. 1.

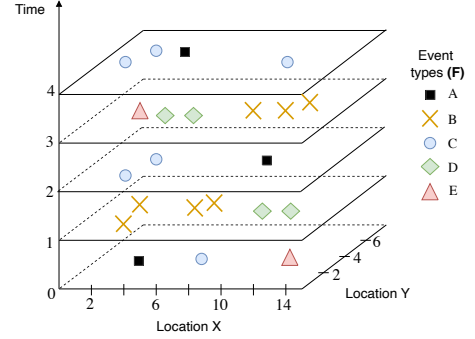


Figure 1: The example data stream of ST event instances of different types represented as a cube.

3.2 The Architecture of the Proposed Framework

In our framework, the dataset D is used to train the proposed neural network, while the set of event types F is used to define the network's architecture. Specifically, the spatial region covered by the event instances in D is partitioned into a grid of $m_1 \cdot m_2$ rectangular cells, where m_1 and m_2 are user-specified parameters. At the center of each cell, we place a group of neurons, each corresponding to an event type in F . Within each cell, all neurons are fully connected via bidirectional synapses. That is, for any two neurons n_i and n_j in the same cell, there exist synapses $n_i \rightarrow n_j$ and $n_j \rightarrow n_i$. We also introduce bidirectional synapses between neurons belonging to different cells whose centers are within a user-defined spatial threshold.

Formally, let the spatial location of neuron n_i at the center of cell i be denoted by $n_i.loc$, and let $n_j.loc$ represent the location of neuron n_j at the center of cell j . A synaptic connection is established between n_i and n_j if and only if

$$\text{dist}(n_i.loc, n_j.loc) \leq \delta,$$

where δ is a user-specified distance threshold.

An example of the resulting network architecture is illustrated in Fig. 2. By $\mathcal{N}$ we symbolically denote the constructed neural network. Thus, the proposed network can be compared to reservoir neural networks [11] with recurrent connections between spiking neurons.

3.3 Neuronal Model

Research literature offers many different types of spiking neuronal models. To the most popular belong Leaky-Integrate-Fire (LIF) [5] in which an internal state of a spiking neuron is expressed by a membrane potential value. Such potential is accumulated with spikes incoming to the neuron. After reaching an adequate potential threshold, the LIF neuron emits a spike resetting the membrane potential and enters a short refractory period. The literature offers many extensions to the LIF model: e.g., probabilistic [7] or non-linear LIF [13].

Specifically, each neuron corresponds to a pair consisting of an event type and a spatial cell. Let n denote a neuron assigned to

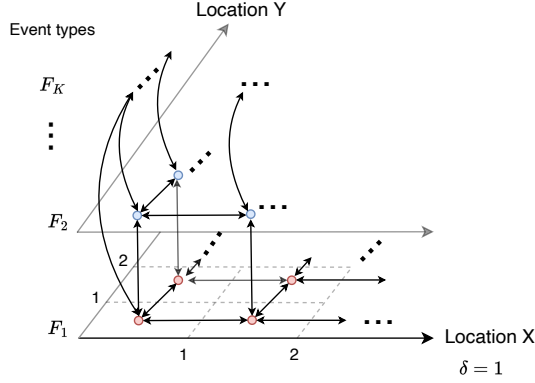


Figure 2: The architecture of the proposed solutions - at each cell neurons of different types are connected using bidirectional synapses. Neurons in the mutual spatial distance no greater than δ are also connected using bidirectional synapses.

event type F . Neuron n emits a spike at time t if there exists an event instance

$$e \in \mathbf{D}$$

such that

$$e.F = F$$

and the spatial location $e.loc$ belongs to the spatial cell represented by neuron n . Thus, spikes are generated directly by event occurrences in the input stream. In our approach each neuron in the network can be both:

- An input neuron emitting spikes.
- An internal neuron receiving spikes from pre-synaptic neurons.

The emitted spikes are subsequently used by the proposed Hebbian-inspired learning rule to adapt synaptic weights between neurons.

3.4 Hebbian-Inspired Unsupervised Learning

Initially, the weight of each synapse in the network is set to zero: $w_{ij} = 0$ (being also a minimal value for any weight) and there is a direct synaptic connection between n_i and n_j . During the learning process, the weights of the network are changed. Specifically, we propose to change weights of synapses according to a modified Spike-Time Dependent Plasticity (STDP) rule and modify the model proposed in [4]. The weight w_{ij} of a synapse is increased based on a timing of spikes emissions of neurons n_i, n_j . Let assume that n_j emits a spike at time t and such emissions follows emission of a spike by n_i . The change of synapse's weight between n_i, n_j is calculated according to Eq. 1.

$$\frac{dw_{ij}(t)}{dt} = \eta (A(t_j) - R(t_j)w_{ij}(t)) - \lambda w_{ij}(t), \quad (1)$$

where $A(t), R(t)$ are defined as follows:

$$\frac{dA(t)}{dt} = -\frac{1}{\tau_A} A(t), \quad \frac{dR(t)}{dt} = -\frac{1}{\tau_R} R(t). \quad (2)$$

In Eq. 1, we incorporated three main components:

- (1) $A(t)$ represents a *short-term* excitatory component of the synaptic weight. Following [4], each time the presynaptic neuron n_i fires, $A(t)$ is set to 1 and subsequently decays exponentially with time constant τ_A . Intuitively, if the post-synaptic neuron n_j fires shortly after n_i , then $A(t)$ remains close to 1, resulting in a larger increase in the synaptic weight w_{ij} . Conversely, if there is a longer delay between the firing of n_i and n_j , the value of $A(t)$ decreases, leading to a smaller weight update. The parameter η is a user-defined learning rate (e.g., 0.01).
- (2) $R(t_j)$ represents a *short-term* inhibitory and scaling component. Each time the postsynaptic neuron n_j fires, $R(t)$ is set to 1 and exponentially decays with the control parameter τ_R . This component performs a scaling role allowing to suppress the uncontrolled weight grow. In terms of ST pattern discovery, this component plays a balancing role: it allows reduce the significance of patterns of most common event types in the data stream $\mathbf{D}$.
- (3) $\lambda w_{ij}(t)$ is a natural *long-term* weight decay component. It allows us to adapt the proposed architecture to learning from continuous data stream: over time the weights of unused synapses tend to vanish, which expresses the changing memory of the architecture and fluctuations of ST patterns in the environment. λ is a user-given decay parameter. Larger λ induces faster decay, while for smaller λ decay is slower. Usually, $\lambda \ll \tau_A, \tau_R$.

Based on Eq. 1, one can establish the maximal bound w_{max} on w_{ij} . Such a maximal bound helps to select the minimum threshold of weights in order to mine only significant ST patterns.

THEOREM 3.1. *The upper-bound w_{max} on any synaptical weight w_{ij} is equal to $\frac{\eta}{\lambda}$.*

PROOF. Let us rewrite Eq. (1) as follows:

$$\frac{dw_{ij}}{dt} = \eta A(t_j) - (\eta R(t_j) + \lambda) w_{ij}, \quad (3)$$

As stated above, we have the following assumptions $A(t) \geq 0$ and maximum of $A(t)$ is $A_{max} = 1$, $R(t) \geq 0$, $\lambda > 0$ and $\eta > 0$. Thus, the following inequality is preserved:

$$\eta R(t) + \lambda \geq \lambda. \quad (4)$$

By rewriting Eq. (3) and applying Inq. (4) the following upper bound on $\frac{dw_{ij}}{dt}$ is preserved:

$$\frac{dw_{ij}}{dt} \leq \eta A_{max} - \lambda w_{ij}, \quad (5)$$

which can be further simplified by replacing A_{max} with 1:

$$\frac{dw_{ij}}{dt} \leq \eta - \lambda w_{ij}. \quad (6)$$

Assume that $w_{max} = \frac{\eta}{\lambda}$, then we have:

$$\frac{dw_{ij}}{dt} \leq \eta - \lambda w_{ij} \leq \eta - \lambda w_{max} = 0. \quad (7)$$

Thus, if at any time w_{ij} exceeds $w_{max} = \frac{\eta}{\lambda}$, its derivative $\frac{dw_{ij}}{dt}$ becomes negative, causing w_{ij} to decrease. $\square$

3.5 Rule Definitions

So far, we have presented learning rules for individual synapses in the network. We now formally introduce the definition of an STDTR and present the learning and pattern discovery algorithms.

Definition 3.2 (Spatio-Temporal Dependency Rule, STDTR). An STDTR is a pair of tuples

$$\vec{r} = (F_1, loc_1) \xrightarrow{w_{12}} (F_2, loc_2),$$

where each tuple (F_i, loc_i) corresponds to the event type and spatial location associated with neuron n_i , and w_{ij} denotes the synaptic weight between the presynaptic neuron n_i and the postsynaptic neuron n_j . For an STDTR $\vec{r}$, its significance $\text{sig}(\vec{r})$ is defined as the synaptic weight value of a synapses constituting $\vec{r}$.

Our Definition 3.2 differs from similar definitions presented in, e.g., [6], in that it explicitly accounts for the spatial location of patterns, while the strength of dependency between event types is represented by synaptic weights.

Definition 3.3 (Significant STDTR). We call an STDTR $\vec{r}$ significant if its support $\text{sig}(\vec{r})$ is greater than or equal to the threshold $\theta \cdot w_{\max}$.

The definition of significance follows approaches widely used in the literature [6, 12, 14, 15]. The threshold ratio θ should satisfy $\theta \in (0, 1)$.

Let us denote the snapshot of all synapses $(n_i, n_j) \in \mathbf{W}$ that at a given time t have a synaptic weight w_{ij} greater than or equal to threshold $\theta \cdot w_{\max}$ by $\mathbf{W}$. By Definition 3.2, set $\mathbf{W}$ constitutes the set of all significant STDTRs at time t : $\mathbf{R}$.

4 Algorithms

In this section, we present the online Algorithm 1 for mining all significant STSPs. The algorithm processes the event stream $\mathbf{D}$ sequentially. For every incoming event instance $e \in \mathbf{D}$, the corresponding neuron n_t emits a spike, synaptic weights are updated according to the proposed Hebbian-inspired rule, and current significant STSPs are mined and displayed to the user. The proposed algorithm takes a lazy approach: only weights that currently are in $\mathbf{W}$ and those affected by spiking of n_t are updated in when processing event e . Such approach allows us to avoid computationally heavy updates of all synaptic weights in network $\mathcal{N}$. In Section 5, we prove that such lazy approach is theoretically correct.

Algorithm 1, in addition to stream $\mathbf{D}$, requires a few user-given parameters necessary to construct neural network $\mathcal{N}$ and parameters of unsupervised learning. For each event e , the algorithm runs a sub-procedure given in Algorithm 2 that performs a local lazy update of synaptic variables and returns the set $\mathbf{M}$ of synapses needed to obtain current maximal significant STSPs. $\mathbf{M}$ consists of all synapses in $\mathbf{W}$ obtained from previous iteration of Algorithm 1, as well as of all synapses of pre or post synaptical neurons of the firing neuron n_t : S_t^{in} , S_t^{out} . For each such synapse, the traces A_{ij} and R_{ij} , as well as the synaptic weight w_{ij} , are decayed according to the elapsed time since the last update. Then, incoming synapses from neurons that fired before n_t are strengthened according to the learning rules presented in Section 3, while outgoing synapses from n_t have their activation trace set to one.

Algorithm 1: Online STSP Mining

Input: Event stream $\mathbf{D}$, grid size $m \times n$, spatial threshold δ , learning parameters $\eta, \lambda, \tau_A, \tau_R$, significance ratio θ
Output: Set of maximal significant STSPs $\mathbf{P}$

- 1 Initialize network $\mathcal{N}$ according to $\mathbf{F}$, $m \times n$, and δ ;
- 2 Set all synaptic weights $w_{ij} \leftarrow 0$;
- 3 Set all traces $A_{ij} \leftarrow 0$, $R_{ij} \leftarrow 0$;
- 4 Set $t_{ij}^{\text{last}} \leftarrow 0$ for every synapse (n_i, n_j) ;
- 5 Set $w_{\max} \leftarrow \frac{\eta}{\lambda}$; // According to Thm. 3.1.
- 6 Set $\mathbf{W} \leftarrow \emptyset$; // Initialize synapses' set $\mathbf{W}$.
- 7 **foreach** incoming event instance $e_t \in \mathbf{D}$ **do**
- 8 $c \leftarrow$ cell containing e_t .loc;
- 9 $n_t \leftarrow$ neuron assigned to event type e_t .F in cell c ;
- 10 $t \leftarrow e_t$.time;
- 11 $\mathbf{M} \leftarrow \text{LazyWeightUpdate}(n_t, t, \mathbf{W}, \mathcal{N})$;
- 12 **foreach** synapse $(n_i, n_j) \in \mathbf{M}$ **do**
- 13 **if** $w_{ij} \geq \theta \cdot w_{\max}$ **then**
- 14 Update or Add (n_i, n_j) in $\mathbf{W}$;
- 15 **else**
- 16 Remove (n_i, n_j) from $\mathbf{W}$ if present;
- 17 Display($\mathbf{R}$) being the set of all STDTRs induced by $\mathbf{W}$;

Algorithm 2: LazyWeightUpdate

Input: Spiking neuron n_t , current time t , significant synapse set $\mathbf{W}$, network $\mathcal{N}$, parameters $\eta, \lambda, \tau_A, \tau_R$
Output: Set $\mathbf{M}$ of updated synapses

- 1 $S_t^{\text{in}} \leftarrow \{(n_i, n_t) : (n_i, n_t) \in \mathcal{N}\}$;
- 2 $S_t^{\text{out}} \leftarrow \{(n_t, n_j) : (n_t, n_j) \in \mathcal{N}\}$;
- 3 $\mathbf{M} \leftarrow S_t^{\text{in}} \cup S_t^{\text{out}} \cup \mathbf{W}$;
- 4 **foreach** synapse $(n_i, n_j) \in \mathbf{M}$ **do**
- 5 $\Delta t \leftarrow t - t_{ij}^{\text{last}}$;
- 6 $A_{ij}(t) \leftarrow A_{ij}(t_{ij}^{\text{last}}) \exp\left(-\frac{\Delta t}{\tau_A}\right)$;
- 7 $R_{ij}(t) \leftarrow R_{ij}(t_{ij}^{\text{last}}) \exp\left(-\frac{\Delta t}{\tau_R}\right)$;
- 8 $w_{ij}(t) \leftarrow w_{ij}(t_{ij}^{\text{last}}) \exp(-\lambda \Delta t)$;
- 9 $t_{ij}^{\text{last}} \leftarrow t$;
- 10 **foreach** incoming synapse $(n_i, n_t) \in S_t^{\text{in}}$ **do**
- 11 **if** n_i fired before n_t **then**
- 12 $w_{it} \leftarrow w_{it} + \eta (A_{it}(t) - R_{it}(t) w_{it})$;
- 13 $w_{it} \leftarrow \max(0, w_{it})$;
- 14 $R_{it}(t) \leftarrow 1$;
- 15 **foreach** outgoing synapse $(n_t, n_j) \in S_t^{\text{out}}$ **do**
- 16 $A_{tj}(t) \leftarrow 1$;
- 17 **return** $\mathbf{M}$;

5 Analysis

5.1 Correctness of Algorithms

We become analysis with Theorem 5.1 stating that for every event $e \in \mathcal{D}$ and its occurrence time, $e.time$, Algorithm 1 provides the complete set of all significant STDs. Theorem 5.1 shows the correctness of Algorithm 1 under the assumption that synapses' weights not affected by any firing of neuron n_t can only decay over time and that the algorithm provide the exact set of all significant STDs.

THEOREM 5.1. *After processing each event instance $e_t \in \mathcal{D}$, Algorithm 1 outputs the complete set $\mathbf{R}$ of significant STDs induced by the current synaptic weights.*

PROOF. We prove the claim by induction over the processed event instances.

Before any event is processed, all synaptic weights are initialized to zero. Hence, no synapse satisfies

$$w_{ij} \geq \theta \cdot w_{\max},$$

and the algorithm correctly initializes $\mathbf{W} = \emptyset$. Therefore, no significant STD exists and $\mathbf{R} = \emptyset$.

Assume that after processing event instance e_t , the set $\mathbf{W}$ contains exactly all synapses satisfying

$$w_{ij} \geq \theta \cdot w_{\max}.$$

Consider the next event instance e_{t+1} and let n_{t+1} be the corresponding spiking neuron. Algorithm 2 constructs

$$\mathbf{M} = \mathbf{S}_{t+1}^{in} \cup \mathbf{S}_{t+1}^{out} \cup \mathbf{W}.$$

By the learning rules, only incoming synapses of n_{t+1} may increase their weights, while synapses that were previously significant may become non-significant due to decay. All such synapses are contained in $\mathbf{M}$. Therefore, every synapse whose significance status may change during the processing of e_{t+1} is checked by Algorithm 1.

For each synapse $(n_i, n_j) \in \mathbf{M}$, the algorithm keeps it in, or adds it to, $\mathbf{W}$ if

$$w_{ij} \geq \theta \cdot w_{\max},$$

and removes it otherwise. Synapses outside $\mathbf{M}$ were not significant in the previous iteration and receive no weight-increasing update in the current iteration; hence, they cannot become significant. Thus, after processing e_{t+1} , $\mathbf{W}$ again contains exactly all significant synapses.

By Definition 3.2, each significant synapse induces exactly one significant STD, and every significant STD is induced by such a synapse. Consequently, the displayed set $\mathbf{R}$ contains exactly all significant STDs at time t .

By induction, the claim holds after every processed event instance. $\square$

5.2 Time and Space Complexity of Algorithms

We skip the initialization steps of Algorithm 1 and focus on the complexity from the perspective of a single event instance, i.e., the main loop of Algorithm 1 together with Algorithm 2. This is especially important because the algorithm is designed for fast processing of incoming instances of the event stream $\mathcal{D}$. Let us use the following notation:

- $N = m_1 \cdot m_2 \cdot |F|$ – total number of neurons in $\mathcal{N}$,

- E – total number of synapses in $\mathcal{N}$,
- $d_{\max}$ – maximum number of incident synapses, incoming and outgoing, of any neuron $n \in \mathcal{N}$,
- $|\mathbf{W}|$ – number of currently significant synapses.

Time complexity. For each event instance $e_t \in \mathcal{D}$, the following operations are performed:

- (1) Finding the target cell c and the corresponding neuron n_t can be done in constant time $O(1)$ by indexing the spatial grid of the network.
- (2) Algorithm 2 constructs the set

$$\mathbf{M} = \mathbf{S}_t^{in} \cup \mathbf{S}_t^{out} \cup \mathbf{W}.$$

The size of this set is bounded by

$$|\mathbf{M}| \leq d_{\max} + |\mathbf{W}|.$$

Therefore, updating traces and weights for all synapses in $\mathbf{M}$ requires

$$O(d_{\max} + |\mathbf{W}|)$$

time. The additional loops over $\mathbf{S}_t^{in}$ and $\mathbf{S}_t^{out}$ are also bounded by $O(d_{\max})$, so they do not change the asymptotic complexity.

- (3) The loop in Algorithm 1 that checks synapses in $\mathbf{M}$ against the significance threshold also requires

$$O(d_{\max} + |\mathbf{W}|)$$

time.

Hence, the overall time complexity of Algorithm 1 for a single event instance is

$$O(d_{\max} + |\mathbf{W}|).$$

Space complexity. Regarding space complexity, the algorithm stores synaptic variables for all synapses in the network, including weights, traces, and last update times. This requires $O(E)$ memory. In addition, the algorithm stores the set of currently significant synapses $\mathbf{W}$, which requires $O(|\mathbf{W}|)$ memory. Since each significant STD is induced directly by one significant synapse, the displayed set $\mathbf{R}$ requires at most $O(|\mathbf{W}|)$ additional space if materialized explicitly.

Therefore, the total space complexity is

$$O(E + |\mathbf{W}|).$$

6 Experiments

6.1 Used Dataset and Implementation

In our experiments, we used the New York City Check-in Dataset [16]. The dataset contains check-ins related to various social, transportation, and work-related activities. It was collected over a period of more than 10 months and contains 227,428 event instances, each assigned a timestamp, a spatial location expressed as longitude and latitude, and an event type. The dataset contains 251 distinct event types and exhibits a strongly imbalanced distribution: the most frequent categories include bars, private homes, offices, subway stations, gyms/fitness centers, and coffee shops, while many categories occur only rarely.

Figure 3 presents the logarithmic density of the geographical locations of events in the dataset. The figure clearly shows that most events are concentrated in the Manhattan Island area. Figure 4

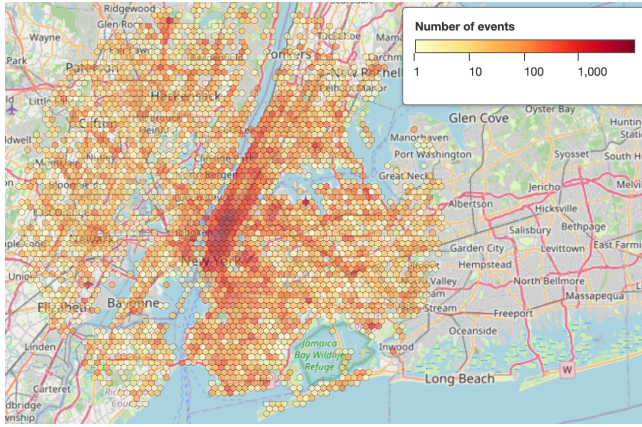


Figure 3: Density of events' locations in the dataset (note logarithmic scale).

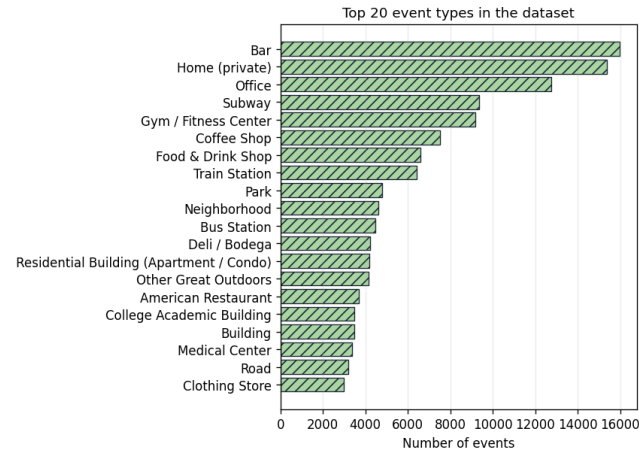


Figure 4: The number of events instances of the top 20 most frequent event types in the dataset.

presents the 20 most frequent event types in the dataset. In our experiments, we aim to discover STSPs that reveal how different types of social activities are related to one another across space and time.

We implemented the proposed approach in Python. The source code is available in the repository at: XXX. The repository also contains the results of all experiments presented below. The accompanying README file provides an overview of the repository structure and includes example usage of the implemented method.

6.2 Quantitative Experiments

We first conducted the quantitative experiment comparing mining times of the proposed approach and other relevant algorithms offered in the literature. Specifically, we compared our approach with those mining spatio-temporal sequential patterns in an offline mode: STS-Miner [6] and STBFM [12]. For each configuration, the event stream was restricted to the most frequent event types and processed up to predefined checkpoints. At each checkpoint,

the proposed online method produced a set of discovered patterns, whose size was used as the reference value for the offline algorithms.

To ensure a comparable output size, STS-Miner and STBFM were run on the same event prefix for different spatial radii R and temporal windows T . For each pair (R, T) , the threshold γ was tuned using binary search to obtain a pattern count as close as possible to the online result:

$$(R^*, T^*, \gamma^*) = \arg \min_{R, T, \gamma} ||P_a(c; R, T, \gamma) - |P_{\text{online}}(c)||.$$

The mining time was then reported for the selected run. A match was treated as acceptable when the pattern-count difference did not exceed 10% of the online result.

The resulting mining-time plots in Figure 5 show the cumulative execution time at consecutive checkpoints for each tested configuration. Across the experiments, the proposed online approach exhibits a stable, nearly linear increase in cumulative time, since each new checkpoint only extends the already processed stream. In contrast, STS-Miner and STBFM are repeatedly executed from scratch on each event prefix after parameter tuning, which leads to higher cumulative costs.

In Figure 6, we present the average Jaccard measure similarity and its standard deviation for the sets of mined patterns in a cross-algorithms comparison. For a checkpoint c , the Jaccard similarity between algorithms was calculated according to Eq. 8. Subsequently, the calculated Jaccard measures

$$J(A_c, B_c) = \begin{cases} 1, & \text{if } A_c \cup B_c = \emptyset, \\ \frac{|A_c \cap B_c|}{|A_c \cup B_c|}, & \text{otherwise.} \end{cases} \quad (8)$$

6.3 Ablation Study

6.4 Qualitative Experiments

7 Discussion and Conclusions

References

- [1] Gowtham Atluri, Anuj Karpatne, and Vipin Kumar. 2018. Spatio-Temporal Data Mining: A Survey of Problems and Methods. *ACM Comput. Surv.* 51, 4, Article 83 (Aug. 2018), 41 pages.
- [2] Berkay Aydin and Rafal Angryk. 2016. Discovering spatiotemporal event sequences. In *Proceedings of the 5th ACM SIGSPATIAL International Workshop on Mobile Geographic Information Systems* (Burlingame, California) (*MobiGIS '16*). Association for Computing Machinery, New York, NY, USA, 46–55. doi:10.1145/3004725.3004735
- [3] Berkay Aydin, Soukaina Filali Boubrahimi, Ahmet Kucuk, Bitu Nezamdoust, and Rafal A Angryk. 2020. Spatiotemporal event sequence discovery without thresholds. *Geoinformatica* (2020), 1–29.
- [4] Marcin Białas, Marcin Michał Mironczuk, and Jacek Mańdziuk. 2020. Biologically plausible learning of text representation with spiking neural networks. In *International Conference on Parallel Problem Solving from Nature*. Springer, 433–447.
- [5] Wulfram Gerstner and Werner M Kistler. 2002. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press.
- [6] Y. Huang, L. Zhang, and P. Zhang. 2008. A Framework for Mining Sequential Patterns from Spatio-Temporal Event Data Sets. *IEEE Transactions on Knowledge and Data Engineering* 20, 4 (April 2008), 433–448.
- [7] Nikola Kasabov. 2010. To spike or not to spike: A probabilistic spiking neuron model. *Neural Networks* 23, 1 (2010), 16–19. doi:10.1016/j.neunet.2009.08.010
- [8] Xiangjie Kong, Menglin Li, Kai Ma, Kaiqi Tian, Mengyuan Wang, Zhaolong Ning, and Feng Xia. 2018. Big Trajectory Data: A Survey of Applications and Services. *IEEE Access* 6 (2018), 58295–58306. doi:10.1109/ACCESS.2018.2873779

Cumulative mining time and baseline vs. online speed-up ratio

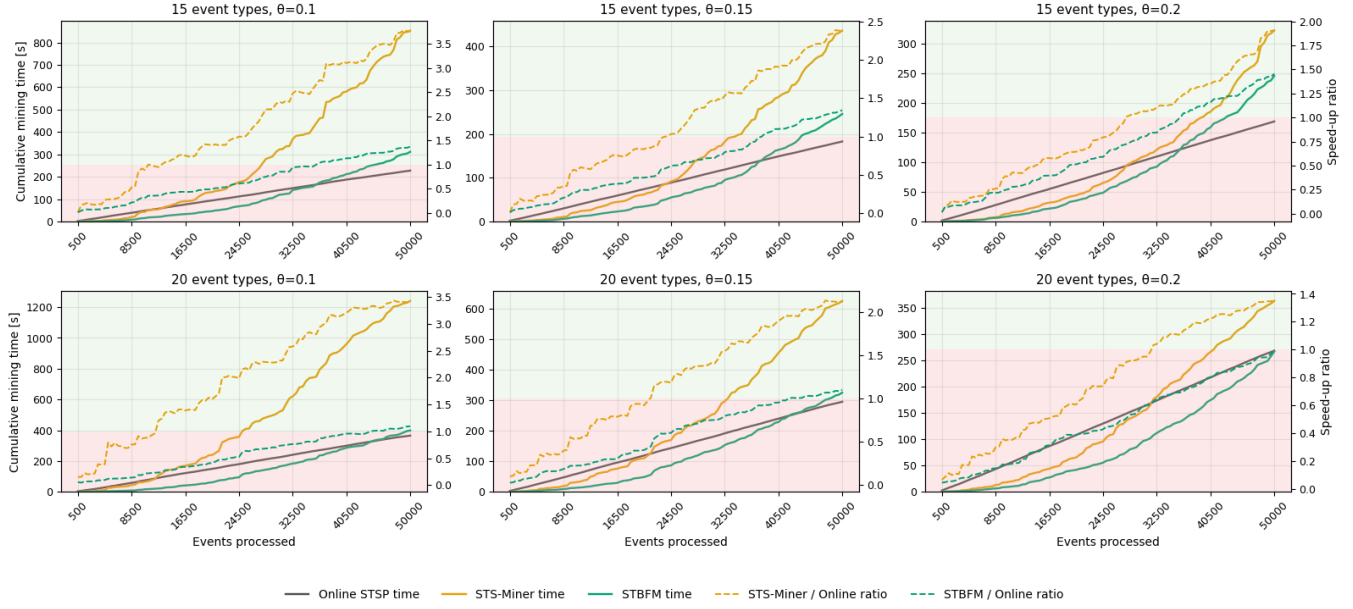


Figure 5: Enter Caption

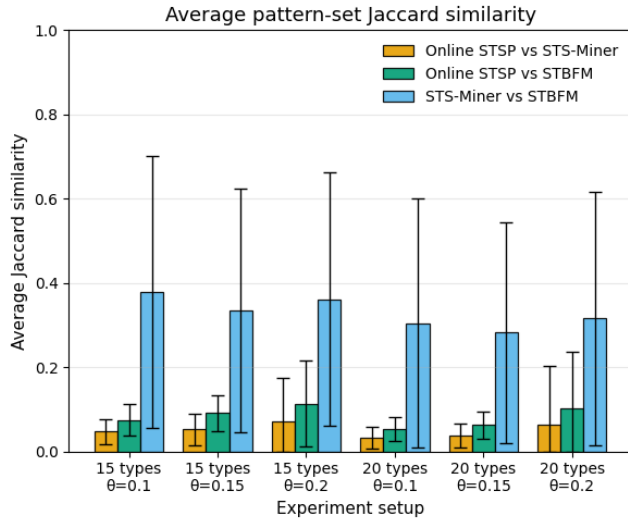


Figure 6: Enter Caption

- [9] Y. Li, J. Bailey, L. Kulik, and J. Pei. 2013. Mining Probabilistic Frequent Spatio-Temporal Sequential Patterns with Gap Constraints from Uncertain Databases. In *2013 IEEE 13th International Conference on Data Mining*. 448–457.
- [10] Zhenhui Li. 2014. *Spatiotemporal Pattern Mining: Algorithms and Applications*. Springer International Publishing, Cham, 283–306.
- [11] Mantas Lukoševičius and Herbert Jaeger. 2009. Reservoir computing approaches to recurrent neural network training. *Computer Science Review* 3, 3 (2009), 127–149. doi:10.1016/j.cosrev.2009.03.005
- [12] Piotr S. Maciag, Marzena Kryszkiewicz, and Robert Bembienik. 2019. Discovery of closed spatio-temporal sequential patterns from event data. In *Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 23rd*

International Conference KES-2019, Budapest, Hungary, 4-6 September 2019. 707–716. doi:10.1016/j.procs.2019.09.226

- [13] Skander Mensi, Richard Naud, and Wulfram Gerstner. 2011. From stochastic nonlinear integrate-and-fire to generalized linear models. *Advances in Neural Information Processing Systems* 24 (2011).
- [14] P. Mohan, S. Shekhar, J. A. Shine, and J. P. Rogers. 2012. Cascading Spatio-Temporal Pattern Discovery. *IEEE Transactions on Knowledge and Data Engineering* 24, 11 (Nov 2012), 1977–1992.
- [15] Sobhan Moosavi, Mohammad Hossein Samavatian, Arnab Nandi, Srinivasan Parthasarathy, and Rajiv Ramnath. 2019. Short and long-term pattern discovery over large-scale geo-spatiotemporal data. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2905–2913.
- [16] Dingqi Yang, Daqing Zhang, Vincent W Zheng, and Zhiyong Yu. 2014. Modeling user activity preference by leveraging user spatial temporal characteristics in LBSNs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45, 1 (2014), 129–142.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009